Week 3 - CloudFormation rollback triggers and failure troubleshooting (UPDATE_ROLLBACK_FAILED and others)

DOP-C02 Domain 2 (Configuration Management and IaC). You must:

1. **Read stack status codes** and know what you can do next.
2. **Control rollback behaviour**: rollback triggers, disabling rollback, retaining resources.
3. **Recover stuck stacks**: `UPDATE_ROLLBACK_FAILED`, `ROLLBACK_COMPLETE`, `DELETE_FAILED`.
4. **Find the root cause** quickly.

Part 1 - Stack status codes and what they allow

Status	Meaning	What you can do
`CREATE_IN_PROGRESS`, then `CREATE_COMPLETE`	Creating / created	Update, delete
`CREATE_FAILED`	Creation failed (seen as stack status when rollback is disabled)	Fix and **update / retry**, **roll back**, or delete
`ROLLBACK_IN_PROGRESS`	Creation failed; deleting what was created	Wait
`ROLLBACK_COMPLETE`	Creation failed and rollback finished; stack is **empty**	**Can't update. Delete and create again** (read the first failure event)
`ROLLBACK_FAILED`	Creation failed **and** deleting created resources failed	**Delete the stack** (retain problem resources if needed), clean up by hand
`UPDATE_IN_PROGRESS`	Update running (also during the rollback-trigger **monitoring period**)	Wait, or `cancel-update-stack` (rolls back)
`UPDATE_COMPLETE_CLEANUP_IN_PROGRESS`, then `UPDATE_COMPLETE`	Update succeeded; old replaced resources being deleted	Update, delete
`UPDATE_FAILED`	Update failed with rollback disabled	Fix and **update / retry**, or `rollback-stack`
`UPDATE_ROLLBACK_IN_PROGRESS`	Update failed; restoring previous state	Wait
`UPDATE_ROLLBACK_COMPLETE_CLEANUP_IN_PROGRESS`, then `UPDATE_ROLLBACK_COMPLETE`	Rollback succeeded; **last working state**	**Can update again** (fix the template first)
`UPDATE_ROLLBACK_FAILED`	Update failed **and the rollback failed**; stuck	`continue-update-rollback` (fix the cause, or `--resources-to-skip`), or delete
`DELETE_IN_PROGRESS`, then `DELETE_COMPLETE`	Deleting / deleted	-
`DELETE_FAILED`	Some resources couldn't be deleted	Fix and **retry delete**, or delete with `--retain-resources`

Status	Meaning	What you can do
`REVIEW_IN_PROGRESS`	Stack from a **CREATE change set** not yet executed	Execute or delete the change set
`IMPORT_IN_PROGRESS` / `IMPORT_COMPLETE` / `IMPORT_ROLLBACK_*`	Resource import	-

- `ROLLBACK_COMPLETE` (after create): **delete and recreate**.
- `UPDATE_ROLLBACK_COMPLETE`: healthy old version, **fix and update again**.
- `UPDATE_ROLLBACK_FAILED`: stuck, `continue-update-rollback`.

Part 2 - Controlling rollback behaviour

- **Create fails:** CloudFormation **deletes everything it created** (`ROLLBACK_COMPLETE`).
- **Update fails:** CloudFormation **restores the previous configuration** (`UPDATE_ROLLBACK_COMPLETE`).
- Rollback is **all or nothing** for the whole operation, including **nested stacks** in the same root operation.

Option	Where	Effect
`--on-failure ROLLBACK` (default), `DELETE`, `DO_NOTHING`	`create-stack`	What happens when creation fails. `DELETE` removes the stack; `DO_NOTHING` keeps resources for debugging
`--disable-rollback`	`create-stack`, `update-stack`, `execute-change-set`	Keep succeeded resources; stack stops in `CREATE_FAILED` / `UPDATE_FAILED`
Console **Preserve successfully provisioned resources**	Stack failure options	Same as disable rollback
After a failure with rollback disabled		**Retry** failed resources, **Update** with a fixed template, or **Roll back** (`aws cloudformation rollback-stack`)
`--retain-except-on-create` / DeletionPolicy `RetainExceptOnCreate`	Stack operation / resource	On rollback, delete **newly created** resources even if DeletionPolicy is Retain (no orphans)
`--timeout-in-minutes`	`create-stack`	Fail creation after N minutes (create only)
`cancel-update-stack`	During `UPDATE_IN_PROGRESS`	Stop the update and roll back

- Debug without repeated rollback and recreation: **disable rollback / preserve resources**, fix, **retry or update**.
- Failed create leaves `ROLLBACK_COMPLETE` and the pipeline can't update: **delete and recreate**; in CodePipeline use action mode `REPLACE_ON_FAILURE`.

Part 3 - Rollback triggers (rollback configuration)

Rollback triggers make CloudFormation **watch CloudWatch alarms** during a create or update and for a **monitoring period afterwards**. If an alarm enters **ALARM**, the **whole operation rolls back**, even if every resource succeeded.

```
aws cloudformation update-stack --stack-name orders-prod \
  --template-body file://template.yaml \
  --rollback-configuration '{
      "RollbackTriggers": [
        {"Arn": "arn:aws:cloudwatch:us-east-1:111122223333:alarm:orders-5xx-high", "Type":
    "AWS::CloudWatch::Alarm"},
        {"Arn": "arn:aws:cloudwatch:us-east-1:111122223333:alarm:orders-latency-composite", "Type":
    "AWS::CloudWatch::CompositeAlarm"}
      ],
      "MonitoringTimeInMinutes": 15
  }'
```

Setting	Values
`RollbackTriggers`	Up to **5** alarms: **CloudWatch alarms or composite alarms** (composite to combine more)
`MonitoringTimeInMinutes`	**0 to 180** minutes **after** all resources are deployed; stack stays `UPDATE_IN_PROGRESS`

- Any trigger in **ALARM** during the operation **or** monitoring period: **rollback**.
- `MonitoringTimeInMinutes: 0`: alarms watched **only while resources deploy**.
- A listed alarm that **doesn't exist** makes the operation **fail**.
- The configuration is **saved on the stack and reused** for later updates; pass an empty list to remove, omit to keep.
- Works for **create, update and change set execution**.
- Typical use: deploy a new Lambda version, task definition or AMI, then watch **5XX / errors / latency** for 15 to 30 minutes.

Rollback triggers vs other automatic rollbacks

Mechanism	Watches	Rolls back
CloudFormation rollback triggers	Alarms during and after a **stack operation**	The **whole stack operation**
CodeDeploy alarms	Alarms during a deployment	The **deployment** (Lambda alias, ECS task set, EC2 revision)
ECS circuit breaker / alarms	Task failures, alarms	The **ECS deployment**
SAM DeploymentPreference	Alarms + hooks (CodeDeploy)	The **Lambda alias shift**; stack update fails
CodePipeline V2 rollback	Failed stage or alarm rule	Back to the last successful **execution**

Part 4 - UPDATE_ROLLBACK_FAILED (the stuck stack)

1. The **update failed**, so CloudFormation started a **rollback**.
2. The **rollback also failed**: some resources couldn't return to their previous state.
3. The stack is **stuck**: no updates; only **continue the rollback** or **delete**.

Common causes

Cause	Example
Resource changed or deleted outside CloudFormation (drift)	Someone deleted the old DB instance or security group by hand
Missing permissions	The **service role** was changed or deleted, or lacks rollback permissions
Old state can't be restored	No capacity for the old instance type, quota reached, dependency gone
Custom resource fails or doesn't respond on the rollback request	Lambda error, no ResponseURL response
Nested stack rollback failed	A child in UPDATE_ROLLBACK_FAILED
Wait condition or signal never arrives during rollback	Instances can't signal

Recovery

```
# 1. Fix the cause if you can (restore permissions, recreate the deleted resource, raise a quota)
# 2. Retry the rollback
aws cloudformation continue-update-rollback --stack-name orders-prod

# 3. If a resource can't be fixed, skip it (its state is then marked as rolled back)
aws cloudformation continue-update-rollback --stack-name orders-prod \
  --resources-to-skip OrdersDbInstance

# Nested stack resource: NestedStackLogicalId.ResourceLogicalId (run on the ROOT stack)
aws cloudformation continue-update-rollback --stack-name orders-root \
  --resources-to-skip AppStack.ServiceSecurityGroup
```

- `continue-update-rollback` retries; when the cause is fixed the stack reaches `UPDATE_ROLLBACK_COMPLETE`. Console: **Stack actions, Continue update rollback**.
- `--resources-to-skip` **marks resources as rolled back without changing them**. Template and real resource **may then differ**: reconcile by hand or with a later update. Only skip **failing** resources; dependants of skipped resources are skipped too.
- **Nested stacks:** run on the **root**, with `ChildLogicalId.ResourceLogicalId`.
- **Last resort: delete the stack** (`--retain-resources` for important resources if deletion fails), then recreate or **import**; contact **AWS Support** if CloudFormation itself is stuck.

Part 5 - Other common failures and fixes

Symptom / error	Likely cause	Fix
`ROLLBACK_COMPLETE` after creation	First resource failure rolled back the creation	Find the **first** `CREATE_FAILED` **event**, fix, **delete and recreate** (CodePipeline `REPLACE_ON_FAILURE`)
`DELETE_FAILED` on an **S3 bucket**	Bucket not empty	Empty it (or custom resource), `DeletionPolicy: Retain`, or retry with `--retain-resources`
`DELETE_FAILED` on **SG / subnet / VPC**	Still **in use** (Lambda ENIs, outside resources, other stacks)	Remove dependencies (wait for Lambda ENIs), retry
Delete blocked on **exports**	Another stack imports an export	Remove imports first
Delete blocked by **termination protection**	Protection on	Turn it off if allowed
Resource already exists	**Custom name** collides with an existing resource	Remove or change the name, or **import** the resource
AccessDenied / not authorized	User or **service role** lacks permissions, including `iam:PassRole`	Fix the role; **CloudTrail** shows the denied call
Requires capabilities	IAM, named IAM, macros	`CAPABILITY_IAM` / `CAPABILITY_NAMED_IAM` / `CAPABILITY_AUTO_EXPAND`
LimitExceeded / quota	Service quota (VPCs, EIPs, instances)	Increase or clean up
Template format error / Circular dependency	Invalid syntax or reference loops	cfn-lint; break the cycle (separate `SecurityGroupIngress`)
Parameter validation failed	Value doesn't exist for an **AWS-specific type** or breaks a constraint	Valid values
No updates are to be performed	Nothing changed	Not a resource error; pipelines often treat it as success
Export cannot be updated as it is in use	Cross-stack lock	Change importing stacks first
Stuck IN_PROGRESS for a long time	Waiting for **signals**, a **custom resource response**, a **wait condition**, or **stabilization** (ECS service never stable)	cfn-init / cloud-init logs, Lambda logs, ECS events; `ServiceTimeout`, **ECS circuit breaker**, shorter timeouts
Embedded stack failed	A nested stack failed	**Child stack's events**
Update fails on a manually changed resource	**Drift**	**Drift detection**, then fix resource or template
Resource handler returned Internal Failure (Registry / third-party)	Extension handler error	Extension's **CloudWatch logs** and execution role

Part 6 - Troubleshooting method

1. **Stack events** (console or `describe-stack-events`): find the **first (oldest)** `*_FAILED` **event** of the operation; its **Status reason** is the **root cause**. Later events are usually "Resource creation cancelled". The console also has a **root cause / timeline view**.
2. **Nested stacks:** open the **child stack** named in "Embedded stack ... failed".
3. **Resource logs:** `/var/log/cfn-init.log`, `cfn-init-cmd.log`, `cloud-init-output.log`; **Lambda CloudWatch Logs** for custom resources and macros; ECS service events and stopped task reasons; extension log groups.
4. **CloudTrail:** the failing API call, caller or role, and error code (AccessDenied, LimitExceeded, ValidationException).
5. **Keep the evidence:** rerun with **rollback disabled** (preserve resources), inspect, then retry or roll back.
6. **Drift detection** if manual changes are suspected.

- **EventBridge** receives **CloudFormation Stack Status Change** and **Resource Status Change** events: route `UPDATE_ROLLBACK_FAILED`, `ROLLBACK_COMPLETE`, `DELETE_FAILED` to SNS / Slack or Lambda.
- Stack `NotificationARNs` (SNS) send all stack events to a topic (older way).
- **CodePipeline:** the CloudFormation action **fails** with the stack; V2 can retry or roll back the stage.

Part 7 - Preventing failed or stuck stacks

Practice	Prevents
Change sets before production updates	Surprise replacements and deletes
cfn-lint / cfn-guard in the pipeline, **Hooks** in the account	Invalid templates, non-compliant resources
Stable service role	Permission failures during updates and **rollbacks**
No manual changes (IAM / SCP) + **drift detection**	UPDATE_ROLLBACK_FAILED from outside changes
Avoid custom names	Already exists errors, blocked replacements
DeletionPolicy / UpdateReplacePolicy on data	Data loss in rollbacks and replacements
Rollback triggers with good alarms	Bad deployments staying live
Custom resources that always respond + `ServiceTimeout`	Hour-long hangs
CreationPolicy + cfn-signal with realistic timeouts	Long waits, false successes
ECS circuit breaker	Stacks waiting hours for unstable services
Dev with rollback disabled, **RetainExceptOnCreate**	Slow debugging, orphaned resources

Part 8 - Exam traps

Trap	Truth
Stack is ROLLBACK_COMPLETE after creation, so update it	**Can't. Delete and recreate** (CodePipeline `REPLACE_ON_FAILURE`)
UPDATE_ROLLBACK_COMPLETE means the stack is broken	**No**, it's at its **last working state** and **can be updated**
UPDATE_ROLLBACK_FAILED: just run update-stack	**Not allowed.** `continue-update-rollback`
Skipping resources keeps everything consistent	Only **marked** rolled back; **reconcile by hand**
Skip a nested resource on the child	Run on the **root** with `NestedStack.Resource`
Watch alarms 30 minutes after an update and roll back automatically	**Rollback triggers** (5 alarms, up to **180** minutes)
Rollback triggers use raw metrics	**CloudWatch alarms or composite alarms**
More than 5 alarms needed	**Composite alarm**
Monitoring time 0 disables triggers	Watches **during** deployment only
Keep resources after a failed create to debug	`--disable-rollback` / `--on-failure DO_NOTHING` / preserve resources
Remove the stack completely when creation fails	`--on-failure DELETE`
Retained resources pile up after failed creates	`RetainExceptOnCreate`
Stop a long-running bad update	`cancel-update-stack` (UPDATE_IN_PROGRESS only)
Where is the real error?	**First FAILED event**, **child stack events**, then **CloudTrail**
DELETE_FAILED on a bucket or SG	Not empty / in use; fix or `--retain-resources`
AccessDenied after an IAM change	**Service role** permissions or `iam:PassRole`
Stuck IN_PROGRESS for an hour	**Signal**, **custom resource response**, or **stabilization**
Alert when stacks enter UPDATE_ROLLBACK_FAILED	**EventBridge** stack status change rule to SNS

Part 9 - Hands-on lab (about 60 to 90 minutes, very low cost)

Lab 1 - ROLLBACK_COMPLETE after a failed create

1. Create a stack with a valid SQS queue and an S3 bucket using a **name that already exists** (for example `aws-logs`).
2. It fails: the queue is **created then deleted**; stack ends in `ROLLBACK_COMPLETE`.
3. Try `update-stack` with a fix: **rejected**. **Delete** and recreate.
4. Find the **first** `CREATE_FAILED` **event** and its reason.

Lab 2 - Preserve resources and retry

1. Create the failing stack with `--disable-rollback` (or Preserve successfully provisioned resources).
2. It stops in `CREATE_FAILED`; the queue remains.
3. Fix the template and **update** (or **Retry**): completes **without recreating the queue**.
4. Optional: `aws cloudformation rollback-stack` on a failed stack.

Lab 3 - Rollback triggers

1. Create a CloudWatch alarm `lab-trigger` on a custom metric `Lab/Errors` (threshold 1).
2. Update a small stack (change a queue tag) with `--rollback-configuration '{"RollbackTriggers":[{"Arn":"<alarm-arn>","Type":"AWS::CloudWatch::Alarm"}],"MonitoringTimeInMinutes":10}'`.
3. During monitoring (stack **UPDATE_IN_PROGRESS**), run `aws cloudwatch put-metric-data --namespace Lab --metric-name Errors --value 5` (or `set-alarm-state --state-value ALARM`).
4. The stack **rolls back**, naming the alarm.

Lab 4 - UPDATE_ROLLBACK_FAILED and continue-update-rollback

1. Create a stack with an `AWS::SSM::Parameter` and other simple resources.
2. Delete the parameter **by hand** (drift), then update the stack so the parameter changes and a sibling resource **fails** (for example an invalid SG CIDR). The rollback can't restore the deleted parameter: `UPDATE_ROLLBACK_FAILED`.
3. `aws cloudformation continue-update-rollback --stack-name lab-rb --resources-to-skip <LogicalId>`: reaches **UPDATE_ROLLBACK_COMPLETE**.
4. **Drift detection** shows the skipped resource as **DELETED / MODIFIED**; fix with an update.

Lab 5 - DELETE_FAILED and notifications

1. EventBridge rule: `source: aws.cloudformation`, `detail-type: CloudFormation Stack Status Change`, status in `[UPDATE_ROLLBACK_FAILED, DELETE_FAILED, ROLLBACK_COMPLETE]`, target an SNS email topic.
2. Stack with an S3 bucket (`DeletionPolicy: Delete`), upload an object, delete the stack: **DELETE_FAILED**, email arrives.
3. Retry with `--retain-resources DataBucket`: stack deleted, bucket remains; empty and delete it by hand.

Clean up

- Delete lab stacks, the alarm, the EventBridge rule, the SNS topic and retained buckets.

Part 10 - Check yourself

Questions

1. Creation failed and the stack is `ROLLBACK_COMPLETE`. Can you update it? What do you do?
2. What does `UPDATE_ROLLBACK_COMPLETE` mean, and can you update the stack?
3. Three causes of `UPDATE_ROLLBACK_FAILED`?
4. How do you recover from `UPDATE_ROLLBACK_FAILED`? What does `--resources-to-skip` do, and the risk?
5. How do you skip a resource inside a nested stack during recovery?
6. What are rollback triggers? How many alarms, and how long can monitoring last?
7. You need more than 5 alarms. What do you do?
8. What happens with `MonitoringTimeInMinutes: 0`?
9. Developers want failed resources kept to see why they failed. Which options?
10. Which `--on-failure` value removes the stack completely when creation fails?
11. How do you stop an update that is clearly going wrong?
12. A CodePipeline CloudFormation action keeps failing because the stack is `ROLLBACK_COMPLETE`. Which action mode fixes it?
13. Which failure event shows the root cause?
14. Stack deletion fails on a security group. Likely reasons and how to finish?
15. How do you get notified when any stack enters `UPDATE_ROLLBACK_FAILED`?

Answers

1. **No. Delete and recreate** after fixing the cause from the first `CREATE_FAILED` event.
2. The update failed and was **rolled back successfully**; **last working state**; **can be updated**.
3. Resources **changed or deleted outside CloudFormation**, **missing permissions** (service role), **old state can't be restored** (capacity, quota, deleted dependency), **custom resource failing**, **nested stack rollback failure**.
4. Fix the cause, then `continue-update-rollback`; if not fixable, `--resources-to-skip` **marks resources rolled back without changing them**. Risk: template and reality **don't match**; reconcile later.
5. On the **root stack**: `--resources-to-skip NestedStackLogicalId.ResourceLogicalId`.
6. **CloudWatch alarms** (or composite) watched during and after a stack operation; **up to 5**, monitoring **0 to 180 minutes**.
7. A **composite alarm** as one trigger.
8. Alarms watched **only during deployment**, no extra period.
9. `--disable-rollback`, `--on-failure DO_NOTHING`, or **Preserve successfully provisioned resources**; then retry, update or roll back.
10. `--on-failure DELETE`.
11. `aws cloudformation cancel-update-stack` during `UPDATE_IN_PROGRESS`.
12. `REPLACE_ON_FAILURE`.
13. The **first (oldest)** `*_FAILED` **event**; for nested stacks, the **child's** events.
14. The SG is **in use** (ENIs from Lambda, outside instances or load balancers, other SG references). Remove dependencies and retry, or `--retain-resources AppSg` and clean up later.
15. **EventBridge rule** on Stack Status Change with status `UPDATE_ROLLBACK_FAILED`, target **SNS** (or Lambda).

Quick memory summary

- **ROLLBACK_COMPLETE** (failed create) = delete and recreate (`REPLACE_ON_FAILURE`). **UPDATE_ROLLBACK_COMPLETE** = fine, update again. **UPDATE_ROLLBACK_FAILED** = **continue-update-rollback** (+ `--resources-to-skip`, root for nested, reconcile). **DELETE_FAILED** = fix or `--retain-resources`.
- **Rollback control:** `--on-failure ROLLBACK / DELETE / DO_NOTHING`, `--disable-rollback` (preserve resources, then retry / update / `rollback-stack`), **RetainExceptOnCreate**, `cancel-update-stack`, `--timeout-in-minutes`.
- **Rollback triggers:** up to **5 CloudWatch / composite alarms**, **0 to 180 min** monitoring, whole operation rolls back, saved on the stack.
- **Troubleshoot: first FAILED event**, child stack events, resource logs, **CloudTrail**, drift detection; notify with **EventBridge** stack status events.